

Floatsat Design Project Report

WS25: Team 1 - Floatsat 2025/2026

Ekin Gürhan (Project Management, Mechanical Design Responsible)

Krunal Rajput (Mechanical Design Responsible)

Finn Krauß (Electrical Design Responsible)

Luis Angel Alvarez Gil (Software Architecture Responsible)

Shahnawaz Hussain (Ground Station Responsible)

Godwin George (Control System Development Responsible)

March 22, 2026

(EG) This report presents the design of a three-axis attitude control demonstrator developed for the Floatsat project. The system emulates basic spacecraft attitude dynamics on a Spherical Air Bearing Unit (SABU) using a reaction wheel-based control architecture. The platform integrates a modular mechanical structure, an embedded control system based on an STM32 microcontroller, an IMU sensor for attitude estimation, and wireless communication with a ground station. Particular attention is given to the mechanical mass distribution to ensure that the center of mass is located close to the center of rotation of the SABU platform. A PID-based control approach is adopted due to its simplicity and robustness for real-time embedded implementation. The resulting system provides a compact experimental platform for demonstrating fundamental spacecraft attitude control concepts.

1. Introduction (EG)

The Floatsat project provides students with practical experience in the design and integration of satellite subsystems through the development of a small attitude control demonstrator. The system is designed to emulate the rotational dynamics of a spacecraft using a three-axis reaction wheel configuration and an onboard inertial measurement unit (IMU). The platform operates on a Spherical Air Bearing Unit (SABU), which allows nearly frictionless rotation and therefore enables the study of spacecraft-like attitude dynamics in a laboratory environment.

The main objective of the project is to design and implement a compact system architecture that integrates the mechanical structure, electrical components, and control system required for basic attitude stabilization and maneuvering.

The presented design focuses on achieving a balanced mass distribution, reliable subsystem integration, and a control approach suitable for real-time embedded implementation.

This report summarizes the final system architecture and the main design decisions taken during the development of the Floatsat demonstrator.

2. Design Overview (EG)

The Floatsat system is designed as a modular platform that integrates mechanical, electrical and control subsystems to demonstrate three-axis attitude control. The architecture is built around three reaction wheels, an inertial measurement unit (IMU) for attitude sensing, and embedded

controllers responsible for system operation and communication.

The STM32 microcontroller executes the real-time control loop and processes sensor data from the IMU, while the ESP32 module manages wireless communication with the ground station. Particular attention is given to practical engineering considerations such as manufacturability, accessibility of components, cable routing, and safe operation on the SABU platform.

A PID-based control approach is adopted due to its simplicity, predictable behavior, and suitability for embedded real-time implementation. This system architecture provides a compact and reliable platform for demonstrating fundamental spacecraft attitude control concepts.

3. System Overview (FK)

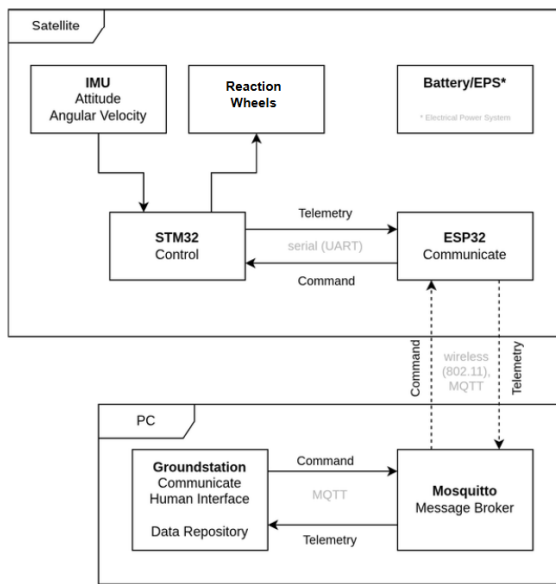


Figure 1: System-level architecture of the Floatsat system

4. System Requirements (FK)

System requirements were derived from the slides and through personal interaction with the instructor. Over the course of the projected

they have been refined multiple times. The final list of requirements is available in the appendix.

5. Mechanical Design (EG)(KR)

This section summarizes the final mechanical configuration of the Floatsat demonstrator and the key design decisions that led from the initial tower concept to the final disc-based architecture. The mechanical design priorities were (i) stable operation on the SABU platform through appropriate mass distribution (CM close to and slightly below the center of rotation), (ii) sufficient structural rigidity within the control bandwidth, and (iii) practical manufacturability and accessibility for integration and testing.

5.1. Design Evolution and Rationale

The first iteration used a vertical three-deck tower architecture. While functionally feasible, this configuration placed the center of mass (CM) significantly above (compared to the final design) the SABU center of rotation (CoR), introducing gravitational torques that reduced passive stability and increased control effort. In addition, the tall aspect ratio reduced rigidity and increased susceptibility to flexural deformation and vibration under reaction wheel excitation.

To address these issues, the design was revised to a compact, horizontally distributed disc platform. The final architecture reduces height-to-width ratio, improves structural stiffness, and enables a mass distribution that places the CM below (or near-coincident with) the CoR.

5.2. Final Mechanical Architecture

The final design consists of a base disc and an elevated electronics deck connected by three cylindrical support rails. Key geometric parameters are:

- Base disc diameter: 193 mm
- Base disc thickness: 5 mm
- Support rail height: 51 mm

All structural parts were manufactured using 3D printing (PLA) and assembled using M2/M3 fasteners with heat-set threaded inserts.

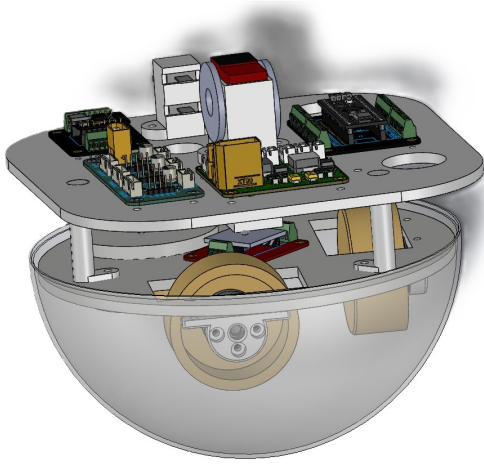


Figure 2: Complete Floatsat mechanical assembly showing the disc-based structure, reaction wheel modules, and the electronics deck.

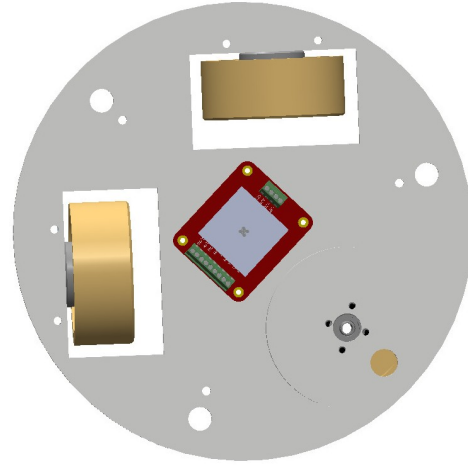


Figure 3: Top view of the lower structural layer showing the placement of the reaction wheel modules.

5.3. Reaction Wheel Mounting and Configuration

Three reaction wheels are mounted to provide torque authority about all principal axes. Each wheel consists of a T-Motor GB2208 brushless motor and a custom flywheel, secured via M3 fasteners into threaded inserts on the base disc. The wheel axes are arranged mutually orthogonal to simplify control allocation and reduce axis coupling effects during multi-axis maneuvers.

5.4. Component Placement and Mass Distribution

Component placement was driven primarily by CM management. The heaviest components (battery and reaction wheel assemblies) are positioned at lower elevation, while lighter electronics are placed on the upper deck. The IMU is mounted close to the geometric center and near the system CM to reduce measurement errors due

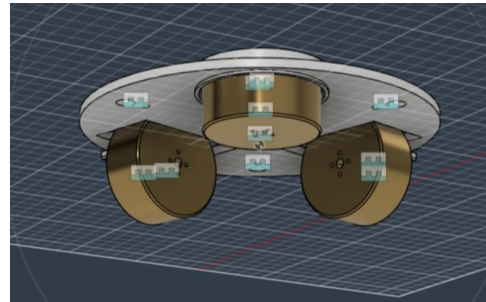


Figure 4: Bottom view of the base disc showing the reaction wheel and flywheel mounting arrangement.

to structural flexure and lever-arm effects during rotation.

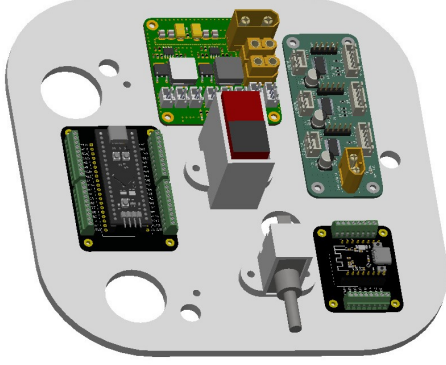


Figure 5: Top view of the upper electronics deck illustrating PCB placement and the central battery location.

5.5. Structural Features and Integration Considerations

The disc geometry improves stiffness by distributing reaction wheel loads across a continuous plate element. The three support rails form a triangular load path, providing symmetric support to the upper deck and reducing torsional warping under asymmetric loading. The vertical spacing between decks provides clearance for cable routing and improves accessibility during integration and inspection.

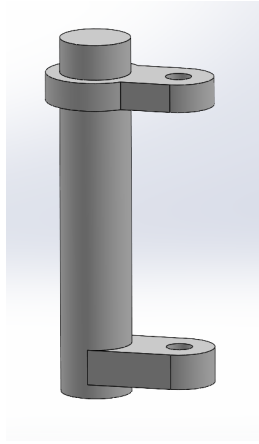


Figure 6: Support rail concept used to connect the base disc and the upper electronics deck (rail height 51 mm).

5.6. Mass Properties

The assembled system mass is 1684.50 g, satisfying the 3 kg course constraint with margin. The mechanical design goal of placing the CM below the SABU CoR was achieved through component layout and verified using CAD mass properties. This configuration improves passive stability and reduces the continuous control effort required to counter gravity-induced torques.

Center of Mass (CM)

CM position in the *Center of Rotation* reference frame:

$$\mathbf{r}_{CM} = \begin{bmatrix} 0.09 \text{ mm} \\ 0.39 \text{ mm} \\ -0.95 \text{ mm} \end{bmatrix}$$

The reference frame and origin are defined consistently with the SABU center of rotation.

Inertia Matrix

Inertia matrix about the center of mass, expressed in the CAD-aligned frame (converted from g mm^2 to kg m^2):

- $J_{xx} = 5.5330 \times 10^{-3}$, $J_{yy} = 5.4291 \times 10^{-3}$, $J_{zz} = 6.4768 \times 10^{-3}$
- $J_{xy} = J_{yx} = -1.1626 \times 10^{-4}$
- $J_{xz} = J_{zx} = -2.6276 \times 10^{-5}$
- $J_{yz} = J_{zy} = 7.8747 \times 10^{-5}$

These values correspond to the final CAD model used for the demonstration.

5.7. Summary of Improvements

Compared to the initial tower concept, the final disc-based design improves mechanical performance and integration quality as follows:

- **CM vs CoR:** CM placed below/near CoR for improved passive stability.
- **Rigidity:** Reduced aspect ratio and higher stiffness, reducing flexural effects.

- **Accessibility:** Open layout improves access to electronics and interfaces.
- **IMU placement:** IMU located near the geometric center and close to the CM.
- **Mass:** Total mass 1684.50 g, below the 3 kg constraint.

6. Electrical Design (FK)

6.1. Components

A set of breakout boards, similar to *arduino-style modules*, are supplied to combine them into the electrical hardware of the satellite. These comprise of:

- STM32F401 *Black Pill* MCU board
- ESP32 C6 *Beetle* SoC board
- 3x *SimpleFOC Mini* motor drivers (on one custom board)
- Sparkfun LSM9DS1 IMU module board
- 3x T-Motor GB2208 (+ Fly Wheel)
- 2x Li-ion batteries (in 2s pack, 7.2V nominal)
- EPS board (battery in/out, 3v3 and 5v step-down converters)
- various consumables & tools (stranded wire, crimp connectors, etc...)

6.2. Wiring

In the first iteration, the wiring was prepared in a way to lay out all components as a flat sat¹ and were later transferred to the static installation in the satellite structure (see block diagram in fig. 7) and fixed using cable ties to avoid a changing moment of inertia. To reduce mass, the supplied power cables with XT connectors were shortened.

¹All satellite electronics and instruments connected on a table, not attached to a structure

6.2.1. Cable color codes

The project uses the usual color codes in electronics, i.e. red: voltage source, black: ground.

For all non-supply wires, other colors are used, preferably always the same in recurring buses and unique colours per bus.

6.3. Power distribution

The EPS² board handles conversion of the battery voltage to stable 3.3V and 5V voltage sources and also provides battery voltage.

The motor specifications declare 3-4s³ batteries (assumably lithium) as reference for its rated voltage [1]. Therefore, the motor is supplied directly with the (3s) battery voltage, as that is the highest voltage available (although it varies slightly depending on charge).

The STM32 and ESP32 are supplied with 5 V, as the quality of the 3.3 V output of the EPS was not assessed and it is assumed that the internal regulators with their additional capacitors may handle additional voltage noise caused by the motors better.

Regarding the IMU board, in a real satellite system it would be perhaps more practical to install the IMU on the same 3.3 V power bus as an STM32, coming from the EPS, but for the sake of simplified development using the USB powerable dev boards, the internal regulator of the black pill board are used to also power the IMU.

6.4. Communication Interfaces

Two serial communication interfaces need to be laid out on the satellite. For communication between the ESP32 and the STM32, UART is used, which transfers serialized command and telemetry data between the microcontroller boards. UART is simple to use and available on both boards.

²Electrical Power System

³batteries in series

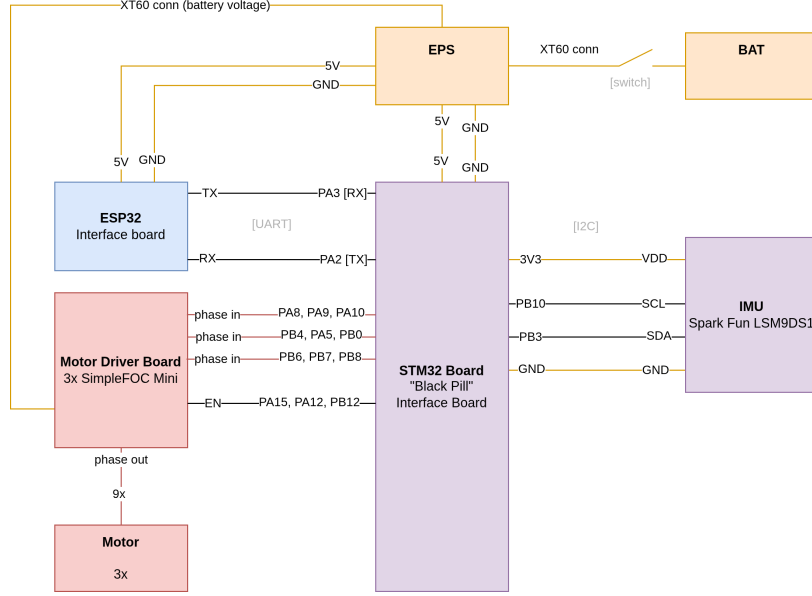


Figure 7: Connection block diagram. Yellow: Power distribution, purple: control-related, red: motor and driver, blue: telemetry/telecommands

For communication between the STM32 and the IMU, I2C is preferred over SPI, as the supplier of the breakout board designed it to be the simplest solution [2].

6.5. Low Level Motor Control

The supplied GB2208 motors are simple 3-phase brushless BLDCs. For testing, a single motor was controlled via SimpleFOC, a library predominantly used to control BLDC motors such as ours [3]. Most use cases seem to revolve around closed loop Field Oriented Control, but as neither current sensing nor HAL sensors are available, we need to control the motor in an open loop fashion and at the same time raise the voltage and thus torque during acceleration phases. For power-stage switching, we utilize the SimpleFOC Mini board (Figure 8). To generate a discretized sine wave, openFOC modulates a sine wave on a PWM duty cycle (Figure 10). Voltage and current and thus excess heat is relative to the total duty cycle and must thus be ramped together with the motor.

During a large portion of the project, we had problems with significant noise development and fast acceleration. While the acceleration has

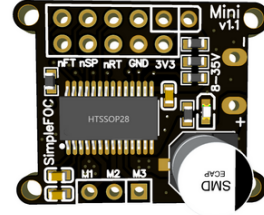


Figure 8: simpleFOC mini board used to drive the motors; an H bridge with 6 MOSFETS.

been fixed ($> 100 \text{ rad/s}^2$) by using a discretized ramp, noise remains a problem, especially at lower speeds.

7. Attitude Estimation (FK)

To control the angular velocity and position of the floatsat, the actual attitude (and its first derivative) of the system must be acquired.

To accomplish that, we were equipped with a 9-DOF MEMS IMU, a device that measures acceleration, angular velocity (gyroscope) using

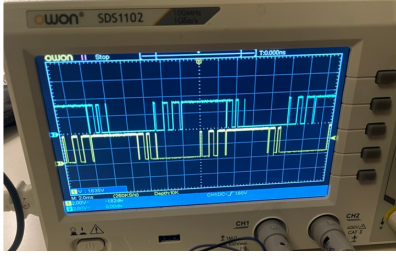


Figure 9: simpleFOC on STM32F4. Two of the 3 phases on oscilloscope

micro-mechanical elements on its chip. Additionally this IMU also measures the earth’s magnetic field, which can be used as a static heading-reference to the coordinate frame that does not accumulate error. [4]

The accelerometer can provide direction of gravity and thus a second fixed reference to the coordinate frame. Accelerometers work well when the system rotates slowly or not at all. When the system is rotating fast, the error of the accelerometer data increases and it may be useful to combine it with gyroscope data, which measures only rotation and is independent of gravity. [5, pp. 195–199]

For this sensor fusion, we decided to use the madgwick filter. It is a popular and fast [6] attitude filter, popular in embedded systems and thus many implementations already exist; we use one of those. However, although the filter works using quaternions, the rest of the system uses euler angles, due to reduced complexity. As the system can physically not completely rotate around pitch and roll axes, gimbal lock is impossible.

7.1. Pipeline

8. Control System Development (GG)

This report documents the control system development for the FloatSat attitude control testbed. The project successfully progressed from basic open-loop control to a fully implemented cascaded PID architecture with proper loop rate separation.

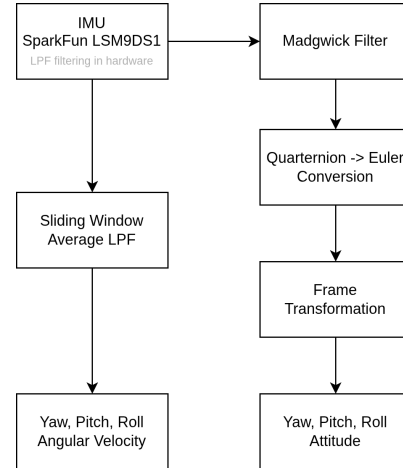


Figure 10:

- **Status:** While the control architecture was successfully implemented and the theoretical framework established, full PID tuning was not completed due to time constraints.
- **Scope:** This section details the control design process, implementation decisions, and the theoretical basis for loop rate selection.

8.1. Starting Point: Open-Loop Control

The project began with a simple open-loop control approach where motor commands were sent without feedback.

- **Initial Phase:** Involved directly commanding motor voltages to the reaction wheels and observing the satellite’s response on the air bearing.
- **Implementation:** Basic motor commands were sent via the ground station, and the satellite’s motion was observed through the telemetry system.
- **Constraint:** No feedback correction was applied during operation.

8.2. Closed-Loop Control Evolution

Following the limitations of open-loop control, the system transitioned to closed-loop architectures:

- **Single-Loop PID:** This represented the first closed-loop control system for the Float-Sat, where motor commands were directly calculated based on attitude error.
- **Final Architecture (Cascaded PID Control):** The project successfully implemented a cascaded (nested) PID architecture with separate inner and outer control loops. This represents the industry standard for spacecraft attitude control systems.

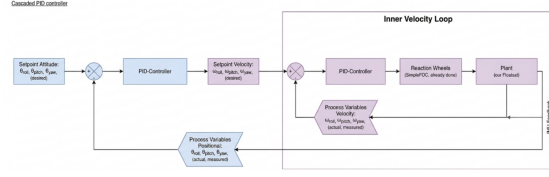


Figure 11: Cascaded PID control architecture illustrating the nested inner velocity loop and outer position loop.

9. Ground Station (SH)

The ground station is responsible for receiving real-time telemetry data from the satellite simulator, visualizing attitude and system state, and sending commands back to the satellite. The communication chain spans from an STM32 flight controller, through an ESP32 wireless bridge, over MQTT to a NestJS backend server, and finally to a Next.js web-based dashboard via WebSockets.

9.1. System Architecture Overview

The ground station system consists of three primary components arranged in a distributed architecture.

8.3. Loop Rate Selection: Theory and Calculation

8.3.1. The 10:1 Ratio Principle

The inner loop speed was set to **500 Hz** and the outer loop to **50 Hz**, maintaining a **10:1 ratio** required for stable cascaded control. These frequencies were selected based on four key factors:

- **Sensor Capacity:** The LSM9DS1 gyroscope has a maximum output of 952 Hz, making 500 Hz a safe choice with a 52% margin.
- **Mechanical Response:** The satellite's mechanical response requires at least 20 Hz minimum; therefore, 500 Hz provides a $25\times$ safety margin.
- **Timer Constraints:** The 1000 Hz base timer requires integer divisors; both 500 Hz and 50 Hz are achievable with dividers of 2 and 20 respectively.
- **Computational Overhead:** The STM32 microcontroller can comfortably handle 500 Hz updates while managing concurrent communication and telemetry tasks.

The 10:1 ratio ensures the inner velocity loop completes 10 updates for every position loop update, preventing control interaction and maintaining system stability.

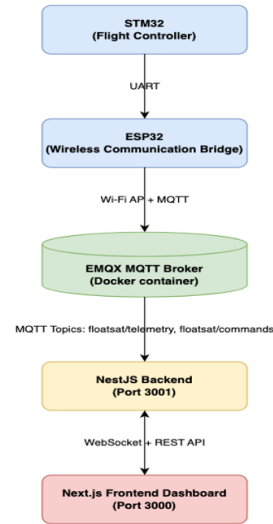


Figure 12: Ground Station System Architecture and Data Flow.

9.2. Communication Flow

The data exchange between the satellite and the operator is divided into two primary logical flows:

9.2.1. Uplink (Telemetry: Satellite → Ground Station)

1. **Data Acquisition:** The STM32 reads IMU sensor data (orientation: roll, pitch, yaw; angular rates: $\omega_x, \omega_y, \omega_z$) and transmits a binary UART frame to the ESP32 containing 6 float values encoded as `uint16` (value $\times 100$).
2. **Processing and Routing:** The ESP32 deserializes the UART frame, constructs a JSON telemetry payload, and publishes it to the MQTT topic `floatsat/telemetry`. The backend subscribes to this MQTT topic, receives the JSON payload, and broadcasts it to all connected frontend clients via WebSocket.
3. **Visualization:** The Frontend dashboard renders the data in real-time: telemetry cards, 3D attitude visualization, and time-series graphs.

9.2.2. Downlink (Commands: Ground Station → Satellite)

1. **Command Initiation:** The operator enters a command on the web dashboard (e.g., set orientation axis to a target angle) and emits the command via WebSocket to the backend.
2. **Transmission:** The backend publishes the command as JSON to the MQTT topic `floatsat/commands` with QoS 1. The ESP32, subscribed to this topic, parses the JSON and translates it into a binary UART command frame sent to the STM32.
3. **Execution:** The STM32 processes the command and actuates the reaction wheels accordingly.

9.3. ESP32 Firmware — Wireless Communication Bridge

9.3.1. Platform and Dependencies

The following technical specifications and dependencies define the operational environment of the ESP32 wireless bridge:

- **MCU:** ESP32-DevKitC (Espressif32)
- **Framework:** Arduino (via PlatformIO)
- **Wi-Fi:** WiFiManager v2.0.17
- **MQTT Client:** PubSubClient v2.8
- **JSON Serialization:** ArduinoJson v7.4.2
- **UART Baud Rate:** 115200 bps [cite: 31]
- **Telemetry Rate:** 100 Hz (10 ms interval)

9.4. Module Architecture

The ESP32 firmware is organized into five modules:

- **wifi_manager** — Starts the ESP32 in Access Point (AP) mode with SSID `ESP32-FloatSat-AP`. The ground station laptop connects to this AP, placing both devices on the `192.168.4.x` subnet. The ESP32 IP is `192.168.4.1`; the MQTT broker runs on the laptop at `192.168.4.2`.
- **mqtt_client** — Manages the MQTT connection to the EMQX broker. Subscribes to `floatsat/commands` for incoming operator commands and publishes telemetry JSON to `floatsat/telemetry`. Uses a 1024-byte buffer, auto-reconnects every 5 seconds on disconnection, and supports up to 10 connection retries at startup.
- **uart_comm** — Implements a state-machine-based UART receiver and command transmitter for communication with the STM32. The protocol uses a custom binary frame format:
 - Start bytes: `0xAA 0x55`
 - Telemetry payload (STM32 → ESP32): 12 bytes ($6 \times \text{uint16}$ little-endian, representing roll, pitch, yaw, $\omega_x, \omega_y, \omega_z$ — each value multiplied by 100)
 - Command payload (ESP32 → STM32): 3 bytes (1 byte command type + 2 bytes `uint16` value)

- CRC-16: CCITT polynomial 0x1021, initial value 0xFFFF, appended as 2 bytes LE

The receiver state machine transitions through: WAIT_START_1 → WAIT_START_2 → RECEIVING_DATA → WAIT_CRC_LOW → WAIT_CRC_HIGH, ensuring framing and integrity validation.

- **telemetry** — Stores the latest telemetry data and serializes it to JSON using `ArduinoJson`. The JSON structure matches the schema expected by the backend:
 - **timestamp:** unsigned long (milliseconds)
 - **orientation:** roll (float), pitch (float), yaw (float)
 - **angularRates:** wx (float), wy (float), wz (float)
 - **reactionWheels:** rw1 (int), rw2 (int), rw3 (int)
 - **power:** voltage (float), battery (int), current (float)
 - **temperatures:** mcu (float), imu (float)
 - **firmware:** string
 - **linkStatus:** packetsPerSec (float), latency (int), lastAck (string), status (string)
- **uart_protocol** — Shared protocol utilities including `floatToU16()` / `u16ToFloat()` conversion ($\text{float} \times 100 \rightarrow \text{uint16}$, clamped to $[0, 360]$), CRC-16 calculation, frame building, and frame verification.

9.5. Command Types

The following commands are defined in the protocol for controlling the satellite:

- **0 (CMD_TYPE_X_ATT):** Set X-axis (Roll) attitude
- **1 (CMD_TYPE_Y_ATT):** Set Y-axis (Pitch) attitude
- **2 (CMD_TYPE_Z_ATT):** Set Z-axis (Yaw) attitude

- **3 (CMD_TYPE_Z_VEL):** Set Z-axis angular velocity

9.6. Backend Server

9.6.1. Technology Stack

The backend infrastructure is built using the following technology stack:

- **Runtime:** Node.js with NestJS v11
- **MQTT Client:** `mqtt.js` v5.14
- **WebSocket:** `Socket.IO` v4.8 via `@nestjs/websockets`
- **MQTT Broker:** EMQX (Docker)
- **HTTP Port:** 3001

9.6.2. Module Structure

The backend is organized as a NestJS application with the following modules:

- **AppModule** — Root module importing `TelemetryModule`.
- **TelemetryModule** — Handles telemetry logic (gateway, controller, service, MQTT client) and exports its providers.

9.6.3. MQTT Client Service (MqttClientService)

This is the core communication bridge between the ESP32 (via MQTT) and the frontend (via WebSocket). Key responsibilities:

- **Connection management:** Connects to the EMQX broker at `mqtt://localhost:1883` with a unique client ID (`groundstation-backend-<timestamp>`), clean session, 5-second reconnect period, and 30-second connection timeout.
- **Telemetry subscription:** Subscribes to `floatsat/telemetry` and publishes incoming messages to an `RxJS Subject<TelemetryData>` observable stream.

- **Command publishing:** Exposes a `sendCommand()` method that serializes command objects to JSON and publishes them to `floatsat/commands` with QoS 1 for guaranteed delivery.
- **Connection monitoring:** Provides `isConnected()` for health checks. Logs reconnection attempts, offline events, and connection closures.

9.6.4. Telemetry Gateway (TelemetryGateway)

A WebSocket gateway that bridges the MQTT telemetry stream to frontend clients:

- **@WebSocketGateway** with CORS enabled for `localhost:3000` and `localhost:3001`.
- **subscribeToMqttTelemetry()** — On initialization, subscribes to the `MqttClientService.telemetrySubject` observable. Each telemetry message is broadcast to all connected WebSocket clients via `server.emit('telemetry', data)`.
- **@SubscribeMessage('command')** — Receives commands from the frontend, forwards them to the satellite via `mqttClientService.sendCommand()`, and emits an `event_log` entry to all clients describing the command result.
- **Command types handled:** `set_orientation` (axis + value), `set_velocity`, `calibrate`, `reset`.

9.6.5. Telemetry Controller (TelemetryController)

Provides REST API endpoints at `/api/telemetry/`:

- **Method:** GET, **Endpoint:** `/api/telemetry/status` — Returns system connection status and uptime.
- **Method:** GET, **Endpoint:** `/api/telemetry/latest` — Returns the latest telemetry snapshot.

- **Method:** POST, **Endpoint:** `/api/telemetry/command` — Sends a command (alternative to WebSocket).

9.6.6. MQTT Broker

The system uses an EMQX broker deployed as a Docker container:

```
docker run -d --name emqx \
  -p 1883:1883 -p 8083:8083 -p 8084:8084 \
  -p 8883:8883 -p 18083:18083 \
  emqx/emqx:latest
```

9.6.7. MQTT Topics

- **Topic:** `floatsat/telemetry` — Direction: ESP32 → Backend, QoS: 0, Description: Real-time telemetry data (100 Hz)
- **Topic:** `floatsat/commands` — Direction: Backend → ESP32, QoS: 1, Description: Operator commands with guaranteed delivery

9.7. Frontend Dashboard — Next.js Application

9.7.1. Technology Stack

The frontend dashboard utilizes the following technologies:

- **Framework:** Next.js v16 (App Router)
- **UI Library:** Material UI (MUI) v7
- **3D Rendering:** Three.js v0.182 via React Three Fiber v9
- **Charts:** Recharts v3.7
- **WebSocket:** Socket.IO Client v4.8
- **Language:** TypeScript

9.7.2. Dashboard Layout

Mission-control style dark dashboard (#1a1a2e) with:

- **Header** — Displays mission title ("CubeSat Mission Dashboard SABU Attitude Simulation - FLOATSAT-01"), real-time clock, connection status indicator (green dot = connected, red = disconnected), and "ESP32 Link" badge.
- **Link Status** — Packets/second, latency (ms), last acknowledged command, nominal status.
- **Orientation** — Roll, Pitch, Yaw (degrees).
- **Angular Rates** — wx, wy, wz (degrees/s).
- **Reaction Wheels** — RW1, RW2, RW3 (RPM).

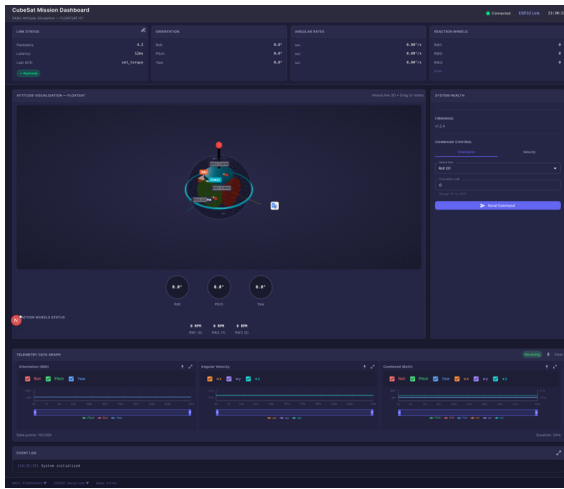


Figure 13: Ground Station Frontend Dashboard.

- **Telemetry Cards Section** — Four telemetry cards in a responsive grid; Left (75%): Interactive 3D attitude visualization; Right (25%): System health panel & command interface.
- **Telemetry Graphs** — Dual real-time line charts; Orientation graph: Roll, Pitch, Yaw over time (degrees, domain $[-180^\circ, 360^\circ]$); Angular velocity graph: wx, wy, wz over time (degrees/s, domain $[-5, 5]$).
- **Graph Features** — Both charts update at 100 Hz, retain 300 data points (3 seconds of history), feature toggle-able series, zoomable brush controls, interactive tooltips, fullscreen modal, and CSV data export.

- **Event Log** — Horizontal scrolling log of timestamped events (commands sent, system messages). Expandable to a fullscreen modal for detailed review.

9.7.3. 3D Satellite Visualization

The `SatelliteVisualization3D` component renders a fully interactive 3D model of the satellite using React Three Fiber and Three.js:

- **Satellite body:** A transparent sphere shell representing the satellite enclosure.
- **Reaction wheels:** Three cylindrical components (X, Y, Z axes) that spin proportionally to their RPM values. Each wheel includes a red marker dot and yellow arrow to visually indicate rotation speed and direction.
- **IMU sensor:** Rendered as a box with a pulsing effect when active.
- **STM32 MCU:** Rendered as a chip with visible pins.
- **Real-time attitude tracking:** The 3D model smoothly interpolates to match the IMU-reported orientation (roll, pitch, yaw) using `THREE.MathUtils.lerp` with Euler order YXZ.
- **Orbit controls:** Users can drag to rotate the camera, scroll to zoom, providing free inspection of the satellite from any angle.
- **Attitude gauges:** Below the 3D view, three circular gauges display numerical Roll, Pitch, and Yaw values with color coding (orange, cyan, green).
- **Reaction wheel status bars:** Show RPM values for all three wheels with color indicating activity.

9.7.4. System Health Panel & Command Interface

The `SystemHealthPanel` component provides:

- System health indicators: Green/red status indicators for STM32, ESP32, and IMU connectivity.
- Temperature display: MCU and IMU temperatures.
- Firmware version display.
- Tabbed command interface with two modes:
 - Orientation mode: Select axis (Roll/Pitch/Yaw), enter target angle (0° – 360°), send `set_orientation` command.
 - Velocity mode: Select axis (Yaw only), enter target angular velocity (-180° to $+180^{\circ}$), send `set_velocity` command.
- Toast notifications: Visual feedback on command success/failure.
- **angularRates:** wx (number), wy (number), wz (number)
- **reactionWheels:** rw1 (number), rw2 (number), rw3 (number)
- **power:** voltage (number), battery (number), current (number)
- **systemHealth:** stm32 (boolean), esp32 (boolean), imu (boolean)
- **temperatures:** mcu (number), imu (number)
- **firmware:** string
- **linkStatus:** packetsPerSec (number), latency (number), lastAck (string), status (string)

9.7.5. WebSocket Hook (useSocket)

A custom React hook manages the entire WebSocket lifecycle:

- Connects to `http://localhost:3001` using Socket.IO with WebSocket transport (fallback to polling).
- Listens for telemetry events, applies angular velocity transformation (values > 180 are negated as $-(v - 180)$ to handle the unsigned uint16 encoding from the ESP32).
- Listens for `event_log` events, maintaining a buffer of the last 50 entries.
- Provides a `sendCommand()` function for the UI to emit commands.
- Emits `subscribe_telemetry` on connection to register interest.

9.7.6. Telemetry Data Types

The TypeScript interface `TelemetryData` provides full type safety across the frontend:

- **timestamp:** number
- **orientation:** roll (number), pitch (number), yaw (number)

9.8. Network Topology and Connectivity

The ESP32 operates in Wi-Fi Access Point (AP) mode, creating its own network:

- **Device: ESP32** — IP Address: 192.168.4.1, Role: Wi-Fi AP + MQTT Publisher/Subscriber
- **Device: Ground Station Laptop** — IP Address: 192.168.4.2, Role: EMQX Broker + Backend + Frontend

This direct AP connection eliminates dependency on external network infrastructure, ensuring reliable low-latency communication during demonstrations.

9.9. Protocol Design

9.9.1. MQTT JSON Protocol (ESP32 ↔ Backend)

Telemetry and command payloads are serialized as JSON over MQTT. The telemetry JSON structure is defined in Section 3.2. Commands follow the format:

```
{
  "type": "set_orientation",
  "data": { "axis": "yaw", "value": 45 }
}
```

9.10. WebSocket Events (Backend ↔ Frontend)

The communication between the backend and frontend utilizes the following WebSocket events:

- **telemetry** — Direction: Server → Client, Payload: Full `TelemetryData` JSON object
- **event_log** — Direction: Server → Client, Payload: { `timestamp`, `message` }
- **subscribe_telemetry** — Direction: Client → Server, Payload: (no payload)
- **command** — Direction: Client → Server, Payload: { `type`, `data` }
- **command_ack** — Direction: Server → Client, Payload: { `success`, `command` }

9.11. Key Design Decisions

1. **ESP32 as AP:** Rather than connecting both devices to an external Wi-Fi network, the ESP32 creates its own AP. This guarantees availability regardless of the lab environment and minimizes latency.
2. **EMQX over embedded broker:** An external EMQX Docker container was chosen over an embedded Node.js broker (Aedes) for reliability, monitoring (EMQX dashboard on port 18083), and scalability.
3. **Reactive pipeline (RxJS Subject):** The backend uses an RxJS Subject to decouple MQTT message reception from WebSocket broadcasting, enabling clean subscription management and backpressure handling.
4. **100 Hz telemetry rate:** The system operates at 100 Hz (10 ms interval) to provide smooth real-time updates for attitude visualization and graphs.
5. **QoS differentiation:** Telemetry uses QoS 0 (fire-and-forget, acceptable for high-frequency data) while commands use QoS 1 (at-least-once delivery) to ensure command reliability.
6. **Dynamic SSR-safe 3D rendering:** Three.js components are dynamically imported (`next/dynamic` with `ssr: false`) to prevent server-side rendering failures, with loading spinners as fallback UI.

9.12. Challenges

1. **100 Hz Throughput** — Sustaining telemetry at 10 ms intervals across UART → MQTT → WebSocket → React without frame drops required memoized components, disabled chart animations, and capped data buffers.
2. **ESP32 AP Mode Constraints** — The ESP32 blocks until a station connects, and the MQTT broker must reside on the laptop at 192.168.4.2. No internet access in AP mode made debugging difficult.
3. **MQTT Broker Migration** — The initial embedded Aedes broker could not handle 100 Hz throughput reliably. Migration to an external EMQX Docker container required restructuring the backend from broker-host to pure client.
4. **Three.js + SSR Conflicts** — Three.js crashes during Next.js server-side rendering. Required dynamic imports with `ssr: false` and Suspense fallbacks for all 3D components.
5. **Cross-Layer Schema Consistency** — Keeping telemetry field names identical across C++ (ArduinoJson), NestJS TypeScript, and React TypeScript — any mismatch caused silent data loss.
6. **Reconnection Resilience** — The wireless link is inherently unstable. Both ESP32 and backend MQTT clients needed robust auto-reconnect logic, and the UI had to reflect connection state in real time.

10. Conclusion of the Ground Station

The ground station system successfully implements a full bidirectional communication pipeline between the FloatSatellite hardware and a web-based operator dashboard. The ESP32 serves as a reliable wireless bridge, translating between the STM32's binary UART protocol and the MQTT JSON protocol used by the ground station software. The NestJS backend efficiently routes telemetry and commands between MQTT and

WebSocket channels. The Next.js frontend provides a comprehensive mission-control-style dashboard with real-time 3D attitude visualization, telemetry graphs at 100 Hz, system health monitoring, and an intuitive command interface for attitude and velocity control.

11. Software Architecture and Inter-Subsystem Communication (LG)

11.1. STM32 Scheduling and ISR Management

The embedded software architecture on the STM32 is designed around a strictly non-blocking, flag-driven scheduling paradigm. The task execution flow is illustrated in Figure 14.

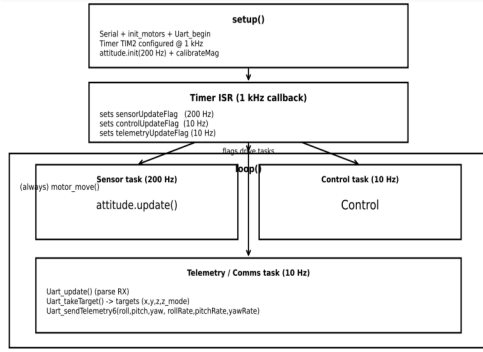


Figure 14: STM32 Task Scheduling Flowchart: setup(), Timer ISR, and loop() tasks.

- **Task Scheduling:** The system utilizes TIM2 as a General-Purpose Timer to generate a 1 kHz interrupt for task scheduling. TIM1 was intentionally avoided for this purpose, as it is an Advanced-Control Timer highly specialized for PWM generation and motor control; reserving it ensures no interference with the reaction wheel actuation.
- **Operating Frequencies:** While initial design iterations considered executing control and telemetry tasks at lower frequencies (e.g., 10 Hz), the final system operates the telemetry transmission at 100 Hz (10 ms interval). This higher frequency is critical to providing smooth, real-time updates for the

3D attitude visualization and graphs on the ground station.

- **ISR Safety and Sensor Acquisition:** The TIM2 Interrupt Service Routine (ISR) is kept deliberately minimal. It solely sets execution flags (e.g., sensorUpdateFlag, controlUpdateFlag, telemetryUpdateFlag) rather than executing heavy instructions. Reading the IMU sensor typically relies on I2C or SPI protocols, which may wait for hardware flags, utilize secondary interrupts, or implement timeouts. Executing these within an ISR is unsafe, as it can cause the processor to hang, introduce severe jitter, or block the UART RX interrupts, leading to dropped communication packets.

Consequently, the actual attitude.update() is executed within the main loop() alongside a free-running timer (such as micros()) to accurately calculate the integration time step.

11.2. UART Communication Interface (STM32 ↔ ESP32)

To facilitate robust inter-board communication between the STM32 and the ESP32 wireless bridge, an asynchronous UART serial link is employed. The communication flow and frame structures are detailed in Figure 15.

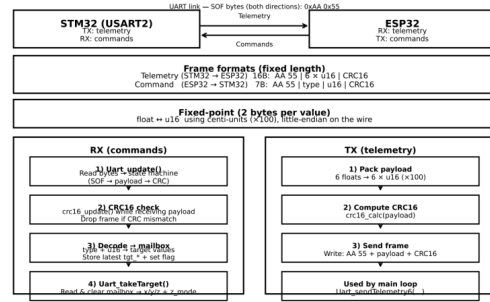


Figure 15: UART Communication Interface: Frame formats, fixed-point encoding, and RX/TX logic flows.

- **Baud Rate and Configuration:** The interface operates at a baud rate of 115200

bps. This provides sufficient bandwidth to transmit the fixed-length telemetry frames at 100 Hz without creating a computational bottleneck, while remaining stable enough to prevent signal degradation over the physical wiring. Development is managed via PlatformIO, which allows for explicit `.ini` configuration of the build environment, including memory and buffer management.

- **Non-Blocking RX State Machine:** The UART reception utilizes the microcontroller’s underlying hardware ring buffer (typically defined as `#define SERIAL_RX_BUFFER_SIZE 64`). A background hardware interrupt continuously populates this buffer. During the main loop execution, a `while(Serial.available())` loop efficiently drains the buffer. This approach pairs with a dedicated state machine (transitioning through states such as `WAIT_START_1`, `WAIT_START_2`, `RECEIVING_DATA`, and `WAIT_CRC_LOW/HIGH`) to parse the incoming bytes one by one. Because the state machine retains its state between cycles, it processes incoming data asynchronously without ever blocking the primary control loop.

11.3. Protocol Design and Data Integrity

Transmitting raw floating-point numbers between different microcontroller architectures can lead to data corruption due to differing memory alignment or endianness. To mitigate this, a custom binary frame format with fixed-point encoding and rigorous error checking was implemented.

- **Fixed-Point Encoding:** Float values are multiplied by 100 and transmitted as 2-byte `uint16` values (little-endian), allowing the system to represent standard integer degrees alongside their fractional components.
- **Error Detection:** Instead of relying on standard UART hardware parity (e.g., `SERIAL_8E1`, which only detects single-bit errors), the protocol integrates a custom CRC-16 checksum (CCITT polynomial `0x1021`). This verifies the integrity of the entire payload, ensuring that if a burst of

noise corrupts multiple bits, the frame is entirely discarded before a dangerous motor command can be executed.

- **Telemetry Frame (STM32 → ESP32):** The fixed-length telemetry payload consists of two start bytes (`0xAA 0x55`), followed by 12 bytes of data representing six `uint16` values (roll, pitch, yaw, and the angular rates ω) and concludes with the 2-byte CRC-16.
- **Command Frame (ESP32 → STM32):** The command payload initiates with the same start bytes (`0xAA 0x55`), followed by a 1-byte command type identifier, a 2-byte `uint16` target value, and the 2-byte CRC-16.
- **Command Allocation:** The command types dictate the target control variable: 0 for X-axis attitude, 1 for Y-axis attitude, 2 for Z-axis attitude, and 3 for Z-axis angular velocity. The STM32 parses these into a central mailbox struct, storing the target coordinates alongside a specific Z-axis mode flag to differentiate between positional and velocity targets.

12. Results

12.1. Ground Station Integration and Testing

To validate the end-to-end communication pipeline, the embedded UART and ESP32 bridging systems were incrementally integrated with the Ground Station. The integration process is detailed as follows:

- **Initial Testing Phase:** Focused on verifying the data link by transmitting dummy payloads—such as a simple “hello” string or values from a random message generator—at a conservative interval of 0.2 seconds (5 Hz).
- **Debugging:** This isolated approach facilitated the debugging of network stability and JSON serialization on the ESP32.
- **Issue Resolution:** During this phase, minor hardware and configuration discrepancies were identified and resolved, specifically regarding incorrect UART pin assignments

and baud rate synchronization issues between the microcontrollers.

- **Final Deployment:** Following these corrections, the full 100 Hz telemetry and command protocols were successfully deployed, resulting in stable, real-time data visualization on the Ground Station dashboard.

13. Conclusion (EG)

This report presented the design and implementation of the Floatsat attitude control demonstrator. The project integrates mechanical, electrical, and control subsystems to emulate basic spacecraft attitude dynamics on the SABU platform using a reaction wheel-based architecture. The final system combines a modular mechanical structure, embedded control hardware based on STM32 and ESP32 microcontrollers, and a simple ground station interface for command transmission and telemetry monitoring. Particular attention was given to mass distribution, subsystem integration, and the practical constraints of the experimental platform. The resulting demonstrator provides a compact platform for studying fundamental spacecraft attitude control concepts and serves as a basis for further experimentation and future improvements.

References

- [1] *T-motor gb2208 specifications website*, <https://store.tmotor.com/product/gb2208-gimbal-type.html>, Accessed: 2025-11-30.
- [2] SparkFun Electronics, *Lsm9ds1 breakout hookup guide*, <https://learn.sparkfun.com/tutorials/lsm9ds1-breakout-hookup-guide/all>, Accessed: 2025-30-11.
- [3] *Simplefoc documentation*, <https://docs.simplefoc.com/>, Accessed: 2025-11-30.
- [4] *Lsm9ds1: Inemo inertial module, 3d accelerometer, 3d gyroscope, 3d magnetometer*, Accessed: 2026-03-21, STMicroelectronics.
- [5] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed. MIT Press, 2017.
- [6] S. O. H. Madgwick, “An efficient orientation filter for inertial and inertial/magnetic sensor arrays,” University of Bristol (UK), Tech. Rep., 2010.
- [7] *Hardwaretimer library - stm32duino wiki*, https://github.com/stm32duino/Arduino_Core_STM32/wiki/HardwareTimer-library, Accessed: 2026-03-20.
- [8] macoclock, *Arduino on the stm32f401cc black pill for mac*, <https://medium.com/macoclock/arduino-on-the-stm32f401cc-black-pill-for-mac-f46ea04fc94b>, Accessed: 2026-03-20.
- [9] *Tbs fusion serial interface specification*, Accessed: 2026-03-20, Team BlackSheep.
- [10] B. Blanchon, *Arduinojson documentation*, <https://arduinojson.org/>, Accessed: 2026-03-20.
- [11] EMQX. “Mqtt on esp32: A beginner’s guide.” Accessed: 2026-03-20. [Online]. Available: <https://www.emqx.com/en/blog/esp32-connects-to-the-free-public-mqtt-broker>

Appendix

A. System Requirements (FK)

Category	#N	Requirement
Functional	F10	The satellite shall be able to rotate along all three axes.
Functional	F20	The control system shall stabilize the satellite attitude autonomously.
Functional	F30	The satellite shall execute manoeuvres upon ground command. Commands: position (pitch, yaw, roll) and angular velocity (yaw), including combinations.
Functional	F40	The ground station (GS) shall allow telemetry monitoring and command transmission via a graphical user interface.
Performance	P10	The control system shall achieve attitude stabilization preferably better than other teams and minimize steady-state error.
Performance	P20	The control system shall achieve attitude stabilization in 30 seconds and maintain it for 30 seconds.
Constraint	C10	The total satellite mass shall not exceed 3 kg. ⁴
Constraint	C20	Only boards made available by the lab shall be used.
Constraint	C30	All mechanical components shall be manufacturable using available tools (3D printer, M2/M3 inserts).
Constraint	C31	All wiring shall follow standard color conventions.
Constraint	C40	Motor control must not destroy motors. ⁵
Constraint	C50	The center of rotation must roughly coincide with the center of mass. The center of mass should be slightly below the center of rotation to prevent toppling.
Constraint	C60	The structure must allow fast access to the battery. The battery should be above a central disk if present.
Constraint	C70	The IMU shall be placed as close as possible (tolerance: 4 mm) to the center of rotation/mass. Deviations may be corrected in software but are undesirable.
Constraint	C80	Reaction wheel motor axes shall be mutually orthogonal. Optionally, the z-axis may align with gravity.
Constraint	C90	The structure must attach securely to the provided bowl and must not slip during operation.
Constraint	C100	The structure must be rigid enough to withstand operation and reaction wheel vibrations.
Constraint	C101	The structure must allow easy disassembly for component reuse.
Interface	Int10	The MCU (STM32) shall provide signals to the motor driver.
Interface	Int20	The IMU shall provide attitude data to the MCU (STM32).
Interface	Int30	The system shall support wireless communication between satellite and ground station.
Interface	Int40	The ESP32 shall provide commands to the STM32 and receive telemetry.
Interface	Int50	A LiPo battery pack shall power the satellite.
Interface	Int60	Each module shall connect via detachable connectors.

⁴Source unclear but assumed requirement.

⁵Mainly concerning overheating.

Category	#N	Requirement
Interface	Int70	Communication between ground station and Floatsat shall use an MQTT broker (pub-sub).
Implementation	Imp20	The electrical design shall minimize cable length and prevent ground loops.
Implementation	Imp30	Embedded software shall separate control logic from communication/UI logic (separate MCUs).
Implementation	Imp40	The structure shall be screw-fastened and modular to allow easy iteration.
Implementation	Imp50	The structure shall include mounts for PCBs.
Implementation	Imp60	The structure shall provide cable fastening (e.g., cable ties), especially for motor wiring.